

Architecting a Unified Narrative Engine: An All-in-One PostgreSQL Design for Integrated RAG and GraphRAG in Python

3.13

This report presents a comprehensive implementation design for integrating both standard Retrieval-Augmented Generation (RAG) and Graph-based RAG (GraphRAG) into the `novelscript-backend` project. The design leverages an "All-in-One PostgreSQL" architecture, utilizing existing dependencies such as LangChain, LangGraph, `pgvector`, and `asyncpg`. It details a multi-stage pipeline for narrative generation, incorporating a sophisticated model routing strategy to optimize performance and cost. The proposed solution focuses on creating a production-ready, operationally simple system that maintains narrative coherence and character consistency by combining semantic and structural knowledge retrieval.

Core Architecture: The All-in-One PostgreSQL Strategy

The foundational design principle for integrating RAG and GraphRAG into the `novelscript-backend` is the adoption of a consolidated "All-in-One PostgreSQL" architecture [1](#) [2](#). This strategic choice moves away from traditional, decoupled systems that rely on multiple specialized databases—such as MySQL for relational data, FAISS for vector search, and Neo4j for graph data—and consolidates all these functionalities within a single PostgreSQL instance [2](#). This approach is explicitly designed to reduce operational complexity and minimize network communication overhead between disparate services [1](#). By leveraging PostgreSQL's extensibility through the `pgvector` extension for vector operations and its native `JSONB` data type and relational capabilities for graph structures, the system achieves a powerful, unified knowledge base managed under one transactional roof [1](#) [2](#). This architectural pattern ensures ACID compliance across all operations involving the novel's evolving knowledge corpus, providing a robust foundation for complex, multi-step narrative generation processes [2](#).

The primary advantage of this strategy is the significant reduction in operational footprint and management burden. Instead of provisioning, configuring, monitoring, and scaling separate external services, the development and deployment teams can focus on a single database environment [2](#). This simplification is particularly valuable in constrained timelines, such as the 72-hour window mentioned in related documentation, where rapid development and deployment are critical [2](#). Furthermore, it eliminates inter-service latency that would otherwise arise from network calls between a main application server and external vector or graph databases. By keeping all data and query logic co-located, the system can execute hybrid queries that seamlessly combine relational filtering, vector similarity searches, and graph traversals, enabling richer and more contextually aware retrievals [2](#). For example, a query could first filter chapters by a specific author using a standard SQL `WHERE` clause, then perform a `pgvector` KNN search on that filtered subset, and finally use the results to traverse a knowledge graph stored in another table—all within a single database session. This unified context is crucial for maintaining the intricate relationships between plot points, characters, and settings throughout a long-form narrative.

However, this consolidation is not without its trade-offs. While PostgreSQL is highly capable, it may not match the raw performance of highly optimized, purpose-built engines like FAISS for massive-scale vector search or Neo4j for deep, wide graph traversals [6](#) [12](#). The performance of `pgvector`, for instance, depends heavily on proper configuration and indexing strategies [6](#) [10](#). Similarly, complex graph traversals using recursive Common Table Expressions (CTEs) in PostgreSQL can become computationally expensive if not carefully managed [2](#). To mitigate these potential bottlenecks, the design incorporates best practices such as the use of advanced indexing schemes like the Hierarchical Navigable Small World (HNSW) index for `pgvector`, which is known for its high-speed approximate nearest neighbor search capabilities [2](#) [8](#). The architecture also relies on efficient asynchronous database access via `asyncpg` to prevent I/O blocking in the FastAPI application, ensuring that the single database connection pool can serve concurrent requests effectively [1](#). The decision to consolidate is therefore a calculated one, prioritizing operational simplicity and architectural cohesion over the theoretical peak performance of specialized, distributed systems. The success of this design hinges on the judicious application of PostgreSQL's advanced features to deliver sufficient performance for the scale and complexity of the `novelscript-backend`'s use cases.

The technical realization of this architecture relies on two key PostgreSQL extensions. First, `pgvector` is used to store and query high-dimensional vectors, replacing the need for an external library like FAISS [1](#). This extension allows for the creation of indexes on vector columns, dramatically accelerating K-Nearest Neighbor (KNN) similarity searches,

which are the cornerstone of standard RAG implementations [2](#) [13](#) . Second, the knowledge graph component replaces Neo4j by using native PostgreSQL data types. Specifically, the JSONB data type is used for flexible storage of node properties and potentially for storing parts of the graph structure itself, while dedicated relational tables (`knowledge_nodes` and `knowledge_edges`) are used to represent the formal graph topology [1](#) [2](#) . This combination allows for a hybrid representation where nodes can have rich, semi-structured metadata stored in JSONB, while the relationships between them are managed with the precision and integrity of relational foreign keys [2](#) . This integrated approach is a testament to PostgreSQL's versatility as a polyglot data platform, capable of serving as a central nervous system for a complex generative AI application. The entire system is built upon a set of core Python dependencies, including `sqlmodel` for declarative model definitions that map to the PostgreSQL schema, `asyncpg` for non-blocking database interactions, and `langchain` and `langgraph` to orchestrate the higher-level logic of the RAG and GraphRAG pipelines [1](#) [2](#) .

Feature	Traditional Multi-Component Architecture	All-in-One PostgreSQL Architecture
Vector Storage/Query	External service like FAISS	Native pgvector extension 1
Knowledge Graph Storage	External graph database like Neo4j	Native JSONB and relational tables (<code>knowledge_nodes</code> , <code>knowledge_edges</code>) 1 2
Operational Complexity	High; requires managing multiple services 2	Low; managed within a single PostgreSQL instance 1
Network Overhead	Significant; inter-service communication required 1	Minimal; all operations co-located in a single database session 1
ACID Compliance	Potentially fragmented across services	Unified and guaranteed across all data types 2
Query Capability	Limited to each service's native query language	Unified SQL with support for vector, JSONB, and graph operations 2

In summary, the "All-in-One PostgreSQL" architecture serves as the unifying principle for the `novelscript-backend`'s RAG and GraphRAG implementation. It represents a pragmatic engineering decision that prioritizes development velocity, operational maintainability, and data cohesion over achieving the absolute maximum performance in every individual component. By thoughtfully applying PostgreSQL's extensive feature set—including `pgvector` for semantic search and its native graph-handling capabilities—the system can construct a rich, unified memory for the narrative generation process. This design choice directly aligns with the project's goals of building a robust yet manageable backend, allowing the focus to remain on the sophistication of the generation logic rather than the complexities of a distributed infrastructure.

Standard RAG Implementation via Vector Search

The implementation of standard Retrieval-Augmented Generation (RAG) within the `novelscript-backend` is centered on leveraging the `pgvector` extension within the PostgreSQL database to create a dynamic and semantically-aware memory for the narrative. This process is integral to maintaining continuity and preventing out-of-character (OOC) behavior by providing the Language Model (LLM) with relevant past context when generating new scenes [1](#) [2](#). The design follows a clear lifecycle encompassing data ingestion, vectorization, storage, and retrieval, all orchestrated within the project's established three-stage pipeline.

During the initial planning stage of the pipeline, the source material for the novel, typically broken down into discrete chapters, undergoes a transformation process. Each chapter's textual content is fed into an embedding model, which converts the natural language into a numerical representation—a high-dimensional vector—in a semantic space [2](#). The choice of embedding model is critical for the quality of retrieval; options like OpenAI's `text-embedding-3-small` or `bge-large-zh` are suggested as they produce effective 1536-dimensional vectors [2](#). This vector encapsulates the meaning and context of the chapter. The resulting vector, alongside the original chapter text and other metadata, is then stored in the database. The recommended schema for this is a dedicated column within the `chapters` table, specifically designed to hold the `VECTOR` data type provided by the `pgvector` extension [2](#). This creates a persistent, indexed archive of the novel's history, where every chapter is a searchable document unit.

The efficiency of retrieving relevant information from this archive is paramount, especially as the novel grows in length. Simply scanning the entire `chapters` table for every generation request would be computationally prohibitive. To solve this, the design mandates the creation of a specialized index on the embedding column. The materials strongly advocate for the use of the Hierarchical Navigable Small World (HNSW) index, which has been supported since `pgvector` version 0.5.0 [11](#). Compared to alternative indexing methods like IVFFlat, HNSW offers superior performance for similarity search, making it the ideal choice for this application [4](#). The HNSW algorithm constructs a multi-layered, graph-based index that enables fast approximate nearest neighbor (ANN) searches [3](#) [8](#). When a new scene needs to be generated, the system computes an embedding for the current prompt or context and uses it to query the HNSW-indexed `chapters` table. The database efficiently identifies the top-K most semantically similar previous chapter fragments based on vector distance metrics like cosine similarity [2](#). Key parameters of the HNSW index, such as `m` (the number of bidirectional links created for each new element), can be tuned to find an optimal balance between retrieval accuracy

(recall) and the size/build time of the index [3](#) . This indexing strategy is crucial for making the RAG mechanism scalable and responsive.

The retrieved context from the vector search plays a vital role in Stage 2 of the pipeline: Scene Generation [2](#) . Before the LLM generates the actual prose, it receives a meticulously crafted prompt. This prompt is augmented with the relevant background information pulled from the RAG system. Specifically, the top-K most similar chapter fragments are fetched from the `chapters` table and formatted into the prompt alongside the direct instructions for the scene [2](#) . This provides the model with immediate access to the narrative's recent history, helping it to remember established character personalities, ongoing plot threads, and environmental details. This prevents the common pitfall of OOC dialogue or actions that stem from the model "forgetting" established facts. For example, if a character was established as timid in Chapter 5, fetching a relevant fragment from that chapter during the generation of Chapter 10 can guide the model to write dialogue that is consistent with that trait. The integration of this RAG memory is a continuous loop: after a new chapter is generated, it is immediately embedded and stored back into the `chapters` table, enriching the memory for future generations [1](#) .

The following table outlines the key components and their functions within the standard RAG implementation:

Component	Technology/Method	Role in RAG Pipeline
Text Source	<code>chapters</code> table (text column)	Stores the raw, unprocessed text of each chapter in the novel. 2
Embedding Model	e.g., OpenAI text-embedding-3-small	Converts chapter text into a numerical vector representation. 2
Storage	<code>pgvector VECTOR</code> column	Stores the generated embeddings in the <code>chapters</code> table for persistence and querying. 2
Indexing	HNSW Index (m parameter configurable)	Accelerates K-Nearest Neighbor (KNN) search on embeddings for fast retrieval. 2 3 8
Retrieval Query	<code>pgvector KNN Search</code> (e.g., <code>ORDER BY embedding <=> query_vector LIMIT K</code>)	Fetches the top-K most semantically similar chapter fragments based on the current context. 2
Integration Point	Stage 2: Scene Generation	The retrieved context is prepended to the LLM prompt to ensure narrative consistency and prevent OOC behavior. 1 2

To implement this, the backend code, likely using `asyncpg` for database access, would follow a sequence of steps. First, it would receive a request to generate a new scene, along with context about the current state of the story. Second, it would generate an embedding for this context. Third, it would execute a `pgvector` KNN search against the `chapters` table to fetch the most relevant prior chapters. Fourth, it would format this

retrieved text and feed it, along with the primary generation task, to the appropriate LLM (likely the faster `DeepSeek-v4-flash` model, routed by the system's strategy [1](#) [2](#)). Finally, once the new scene text is generated by the LLM, the backend would embed this new text, connect to the database using `asyncpg`, and insert the new chapter's text and its corresponding vector into the `chapters` table. This cyclical process of generation, embedding, and storage forms the basis of the RAG-powered memory system. While powerful, it is important to note that this method excels at finding semantically similar *text blocks* but may struggle with more abstract or relational queries that require understanding connections between entities, a limitation that GraphRAG is designed to address.

GraphRAG Implementation via Native PostgreSQL Capabilities

While standard RAG provides a strong foundation for narrative consistency by retrieving semantically similar text blocks, GraphRAG introduces a deeper layer of understanding by modeling and querying explicit relationships between entities. The `novelscript` backend design elegantly implements GraphRAG capabilities entirely within the PostgreSQL database, obviating the need for an external graph database like Neo4j [1](#). This is achieved by structuring the knowledge graph using PostgreSQL's native relational tables and the versatile `JSONB` data type, creating a tightly integrated system where structural and semantic knowledge coexist [2](#).

The core of the GraphRAG implementation lies in a well-defined schema comprising at least two dedicated tables: `knowledge_nodes` and `knowledge_edges` [2](#). The `knowledge_nodes` table serves as the repository for all salient entities within the story universe. These entities can include characters, locations, objects, organizations, or even abstract concepts like "a sense of dread." Each row in this table represents a unique entity and contains columns for a unique identifier, the entity's name, and a `JSONB` column to store a variety of properties. This `JSONB` column is particularly powerful, as it can hold flexible, semi-structured metadata such as personality traits, physical descriptions, affiliations, or historical notes associated with the character [1](#) [2](#). Crucially, each node record also includes its own `VECTOR` embedding, generated from its name and properties. This embedding allows the node itself to be discovered through semantic search, bridging the gap between the structured graph and the unstructured text world [2](#).

The `knowledge_edges` table defines the relationships that bind the nodes together, forming the graph's topology. Each row in this table represents a directed relationship (an edge) from a `source_node_id` to a `target_node_id`. These IDs are foreign keys referencing the primary keys in the `knowledge_nodes` table, ensuring referential integrity. The `knowledge_edges` table also includes a `relation_type` column (e.g., 'is_a_friend_of', 'invented', 'visited') to classify the nature of the connection, and can contain additional metadata columns, such as a `weight` to indicate the strength of the relationship or an `evidence` field containing a reference to the specific piece of source text from which this relationship was derived [2](#) . This schema allows for a rich, multi-faceted representation of the story's knowledge, capturing not just what exists, but how things are connected. The entire knowledge graph, persisted as these two tables within PostgreSQL, becomes the backbone for the system's ability to perform "multi-hop" reasoning [2](#) .

Querying this graph structure is accomplished primarily through SQL, specifically using recursive Common Table Expressions (CTEs). Recursive CTEs are a powerful feature in PostgreSQL that allow a query to refer to its own output, enabling hierarchical or graph-like traversals [2](#) . For example, to gather all secondary friends of a character named "Alice," the system could execute a recursive query starting from Alice's node. The query would first find all nodes directly connected to Alice via an "is_a_friend_of" edge. Then, in the next iteration of the recursion, it would find all nodes connected to those newly found nodes via the same relation type, up to a specified depth limit. This process continues until no more nodes are found or the maximum depth is reached. The result is a comprehensive set of nodes representing Alice's entire friend-of-a-friend network. This capability is a key differentiator of GraphRAG over standard RAG, as it allows the system to answer complex queries about interconnectedness that would be difficult to formulate with vector search alone. For instance, it could determine if two seemingly unrelated characters share a common acquaintance or trace a chain of events that passed through several characters and locations.

The integration of GraphRAG is most impactful during the Scene Generation stage of the pipeline [2](#) . When tasked with writing a scene, the system can issue a series of queries to the `knowledge_nodes` and `knowledge_edges` tables to build a detailed profile of the characters involved. For a scene featuring a confrontation between "Alice" and "Bob," the system would first retrieve their respective node records. Then, it would perform a graph traversal to explore their relationships: Who are Alice's allies and enemies? What is Bob's history with Alice? Have they interacted before? Is there a third party, "Charlie," who is connected to both and might be a point of contention? The information gathered from these graph queries provides a rich, relational context that informs the tone, content, and

potential outcomes of the scene. This context is then woven into the LLM prompt alongside the semantically retrieved chapter fragments from the RAG component, creating a prompt that is deeply informed by both the narrative's history and the intricate web of relationships within its world ². This dual-context approach is a cornerstone of the project's strategy for achieving high-fidelity narrative generation.

The following table summarizes the schema and querying mechanisms for the GraphRAG implementation:

Component	PostgreSQL Object	Columns & Data Types	Purpose
Entities	knowledge_nodes table	id (SERIAL PRIMARY KEY), name (TEXT, UNIQUE), type (TEXT), properties (JSONB), embedding (VECTOR)	Stores nodes for characters, locations, etc., with flexible properties and a semantic embedding for search. ¹ ²
Relationships	knowledge_edges table	source_node_id (INT, FOREIGN KEY), target_node_id (INT, FOREIGN KEY), relation_type (TEXT), metadata (JSONB)	Defines directed relationships between nodes, including the type and any associated data. ²
Graph Traversal	Recursive Common Table Expression (CTE)	Self-referencing SELECT statement in SQL	Queries the knowledge_edges table to traverse the graph up to a certain depth, answering multi-hop questions. ²

It is important to acknowledge the potential performance challenges of recursive CTEs, especially with very large or densely connected graphs ². To manage this, the system must implement safeguards, such as limiting the maximum depth of traversal and ensuring that the knowledge_edges table is properly indexed on (source_node_id, relation_type) and (target_node_id, relation_type) to speed up the lookups at each step of the recursion. Despite these considerations, the use of native PostgreSQL features for GraphRAG provides a compelling path to building a sophisticated narrative engine without introducing external dependencies, fully realizing the vision of a self-contained, all-in-one database architecture.

Hybrid Context Retrieval and Model Routing Strategy

The true power of the novelscript-backend architecture emerges from its ability to fuse the distinct strengths of standard RAG and GraphRAG into a unified, hybrid retrieval mechanism. This approach moves beyond relying on a single source of context and instead provides the LLM with a multifaceted view of the narrative universe, combining semantic similarity with explicit structural relationships. This enriched context is then

delivered to the appropriate model, selected via a strict routing strategy designed to balance computational cost with output quality.

The hybrid retrieval process is executed concurrently during the Scene Generation stage of the pipeline [2](#) . When a new scene is to be written, the system initiates two independent but simultaneous queries against the PostgreSQL database. The first query leverages `pgvector` to perform a K-Nearest Neighbor (KNN) search on the `chapters` table [2](#) . Using an embedding derived from the current scene's prompt, it retrieves the top-K most semantically similar fragments from previously generated chapters. This provides the LLM with a temporal and contextual memory, reminding it of recent events, character states, and environmental details [1](#) [2](#) . The second query operates on the knowledge graph, using a recursive CTE to traverse the `knowledge_nodes` and `knowledge_edges` tables [2](#) . This query gathers relational and structural information about the key entities involved in the upcoming scene—for example, it might retrieve a character's full history of relationships, alliances, and conflicts. The results from both queries—the semantically retrieved text fragments and the structurally queried entity profiles—are then concatenated and formatted into a single, comprehensive context block. This block is prepended to the primary instruction in the final prompt sent to the LLM, arming it with a holistic understanding of both the narrative's flow and the underlying logic of its world.

This hybrid approach creates a synergistic effect. The standard RAG component handles the "what happened" by providing verbatim or paraphrased excerpts from the text, grounding the generation in concrete examples. The GraphRAG component handles the "why it matters" by explaining the relationships and connections that give those events meaning. For instance, GraphRAG might reveal that Character A dislikes Location B because Character B (who is Character A's sibling) died there. Simultaneously, RAG might retrieve a recent chapter fragment describing Character A looking at a picture of Character B with sorrow. Combining these two pieces of context allows the LLM to generate a much more nuanced scene—perhaps Character A visits Location B not just out of random curiosity, but with a specific emotional motivation rooted in the established backstory. This fusion of retrieval paradigms is a key innovation of the design, enabling a level of coherence and depth that neither RAG nor GraphRAG could achieve in isolation.

Complementing this advanced retrieval mechanism is a sophisticated model routing strategy aimed at optimizing both cost and performance [1](#) [2](#) . Instead of using a single, monolithic model for all tasks, the system enforces a strict division of labor between two distinct LLMs: `DeepSeek-v4-pro` for high-complexity tasks and `DeepSeek-v4-flash`

for lightweight generation [1](#) [2](#) . This tiered approach is applied across the project's three-stage pipeline.

Stage	Primary Task	Required Model Capability	Recommended LLM	Rationale
Stage 1: Planning	Global understanding, high-level plotting, knowledge graph extraction.	Complex reasoning, summarization, entity/relation extraction.	DeepSeek-v4-pro 2	High accuracy is critical for establishing a coherent global structure.
Stage 2: Scene Generation	Prose generation, dialogue writing, description.	Fast, fluent text generation with good coherence.	DeepSeek-v4-flash 1 2	Prioritizes speed and cost-effectiveness for repetitive generation tasks.
Stage 3: Optimization	Macro-level consistency checks, fact-checking against the knowledge graph.	Analytical reasoning, error detection, logical inference.	DeepSeek-v4-pro 2	Ensures final quality and correctness, justifying the higher cost.

In Stage 1 (Planning), the powerful **DeepSeek-v4-pro** model is tasked with the computationally intensive work of analyzing the initial input materials, structuring the overall plot, and extracting the foundational knowledge graph by populating the `knowledge_nodes` and `knowledge_edges` tables [2](#) . Its superior reasoning capabilities are essential for building a solid narrative skeleton. In Stage 2 (Scene Generation), the bulk of the creative writing occurs. Here, the faster and more cost-effective **DeepSeek-v4-flash** model takes over, responsible for drafting the actual scenes, dialogues, and descriptions [1](#) [2](#) . This model is less suited for complex analysis but excels at generating fluent, stylistically consistent prose quickly. The hybrid RAG/GraphRAG context retrieved in this stage provides it with the necessary information to maintain coherence. Finally, in Stage 3 (Optimization), the **DeepSeek-v4-pro** model is called upon again to perform a macro-level review of the entire generated script. It acts as a quality assurance agent, checking for logical inconsistencies, factual errors against the knowledge graph, and deviations from the established plot and character arcs [2](#) . This rigorous final check ensures the highest possible quality before the script is considered complete. This deliberate routing strategy is projected to yield a significant cost reduction of approximately 10x compared to using only the **DeepSeek-v4-pro** model for the entire process, demonstrating a pragmatic approach to resource management in a complex AI workflow [2](#) .

Workflow Orchestration with LangGraph

The intricate, multi-stage, and conditional nature of the narrative generation process makes it an ideal candidate for orchestration by a framework that can model workflows as explicit, stateful graphs. LangGraph is identified as the central orchestrator for the `novelscript-backend`, providing the necessary tools to manage the complex logic of the three-stage pipeline (Planning, Scene Generation, Optimization) [7](#). By treating the entire process as a graph, the system gains clarity, modularity, and resilience, transforming a potentially tangled sequence of function calls into a well-defined and easily manageable workflow [7](#).

In the LangGraph paradigm, the entire generation process is represented as a directed graph. Each major step or decision point in the pipeline becomes a "node" in this graph [7](#). Nodes are essentially Python functions that perform a specific operation. For the `novelscript-backend`, there would be at least three primary computation nodes: 1. **Planning Node:** This node executes the logic for Stage 1. It would take the initial user prompt, invoke the `DeepSeek-v4-pro` model to perform global understanding, extract key plot points, and populate the `knowledge_nodes` and `knowledge_edges` tables in the database. Its output would be the high-level plan and the initialized knowledge graph. 2. **Scene Generation Node:** This is the heart of the creative process for Stage 2. It would be responsible for generating one or more scenes. This involves coordinating the hybrid retrieval mechanism: firing off the `pgvector` KNN search and the recursive CTE graph traversal queries to fetch context. It would then assemble the final LLM prompt by combining the primary instruction with the retrieved RAG and GraphRAG context. It would call the `DeepSeek-v4-flash` model to generate the prose and then update the `chapters` table in the database with the new content. 3. **Optimization Node:** This node handles Stage 3. It would take the already-generated script and feed it, along with the full knowledge graph, to the `DeepSeek-v4-pro` model for a thorough consistency check. Its output would be either the validated script or a set of suggested corrections.

The connections between these nodes are the "edges" of the graph, and they define the flow of control [7](#). LangGraph allows for the definition of conditional logic on these edges, which is crucial for handling loops and branching paths. For example, after the Scene Generation node runs, a conditional edge would evaluate whether more scenes need to be written (e.g., based on a counter or a completion flag). If so, the workflow loops back to the Scene Generation node. If not, the edge transitions the workflow to the Optimization node. This dynamic routing capability makes the graph adaptable to varying generation requirements and enables the implementation of iterative refinement cycles without hard-coding complex loops in the application logic.

A critical aspect of LangGraph is its management of a shared "state." This state is typically a Pydantic model or a dictionary that holds all the data that evolves throughout the workflow [7](#). In the `novelscript-backend`, this state object would contain the novel's title, the high-level plot outline, the evolving list of generated chapters, and the ever-growing knowledge graph (which would be represented as lists of nodes and edges or as a serialized graph object). As the workflow progresses from node to node, each node reads from the state to get its inputs and writes its outputs back to the state. For instance, the Planning node would write the initial plot outline to the state. The Scene Generation node would read the current state (including the plan and any previously generated chapters), and then write the newly generated chapter text back to the state's chapters list. This explicit state-passing mechanism makes the data flow transparent and predictable, greatly simplifying debugging and extending the system's functionality in the future.

The integration of LangGraph with the rest of the stack is seamless. The LangGraph nodes themselves would be the primary consumers of the other libraries. They would use `asynccpg` to perform the aforementioned database queries for RAG and GraphRAG context retrieval. They would use `langchain` to interface with the LLM APIs for `DeepSeek-v4-pro` and `DeepSeek-v4-flash`, managing prompts, parsing responses, and handling the model routing logic. The entire LangGraph application would be exposed as a FastAPI endpoint, allowing external clients to start a new generation workflow by posting an initial prompt and state to a designated URL. The graph would then run asynchronously, potentially updating its state and progress in the database, until the Optimization stage is complete. This modular, event-driven architecture, powered by LangGraph, provides a robust and scalable framework for managing the complex, interactive, and stateful process of AI-driven novel generation.

Data Population, Dynamic Updates, and Evaluation

A successful implementation of the RAG and GraphRAG system hinges not only on sophisticated retrieval and generation but also on the reliable population of the knowledge base, its continuous dynamic updates, and a rigorous evaluation framework to measure its effectiveness. These operational aspects are critical for transforming the architectural design into a functional and high-quality product.

The initial population of the knowledge graph is a non-trivial data processing step that must occur during the Planning stage of the pipeline [2](#). Since the source material

consists of unstructured text, the system needs a mechanism to automatically extract entities and their relationships. This process, often referred to as Information Extraction, involves several sub-tasks. First, Named Entity Recognition (NER) is used to identify and classify key entities in the text, such as people, places, and organizations. Second, Relation Extraction (RE) algorithms analyze sentences to identify the semantic relationships between these named entities (e.g., "John gave Mary a book"). This can be achieved using pre-trained models or fine-tuned transformer architectures. Third, the extracted entities and relations must be linked to create a coherent graph. For the `novelscript-backend`, this would involve creating or updating rows in the `knowledge_nodes` and `knowledge_edges` tables. For example, upon encountering the sentence "Sherlock Holmes lived in Baker Street," the system would create `knowledge_node` entries for "Sherlock Holmes" and "Baker Street" (if they don't exist) and an `knowledge_edge` connecting them with a `relation_type` of 'lived_in'. This automated extraction process, likely performed by another LLM call during the planning phase, is the engine that drives the initial construction of the GraphRAG memory.

Once the initial knowledge base is established, the challenge shifts to maintaining its consistency through dynamic updates. Every new piece of narrative generated in Stage 2 introduces new information and potentially alters the understanding of existing elements. Therefore, the system must have a mechanism to atomically update the knowledge graph whenever a new chapter is added. This process mirrors the initial population but is focused on incremental changes. When a new chapter's text is generated, it must be re-processed to identify any new entities, new relationships between existing or new entities, or updated attributes of existing entities. For example, if a new character is introduced, a new node must be created. If an existing character develops a new skill, the `properties` JSONB field of their node must be updated. If two previously unconnected characters meet and form a bond, a new edge must be inserted into the `knowledge_edges` table. This update cycle must be handled with care to preserve data integrity, likely wrapped in a single database transaction to ensure that either all updates succeed or none do, preventing the knowledge graph from becoming corrupted. This continuous cycle of generation followed by knowledge extraction and graph updating is what allows the system to evolve and adapt its understanding of the narrative world over time.

Finally, to ensure the system meets its quality goals, a formal evaluation framework is indispensable. The success of the RAG and GraphRAG implementations cannot be judged solely by qualitative human assessment. A quantitative and qualitative protocol is needed for iterative improvement. The evaluation should focus on several key dimensions: 1. **Coherence and Consistency:** This measures whether the generated text adheres to the established lore. Automated checks can compare generated statements against the

knowledge graph (e.g., verifying that a character's stated birthplace matches their node's properties). For narrative flow, metrics could assess the semantic similarity between consecutive generated chapters to ensure smooth transitions. 2. **Relevance:** This evaluates whether the generated content directly addresses the prompt and contributes meaningfully to the plot. This can be assessed by having the **DeepSeek-v4-pro** model act as an evaluator, scoring the relevance of a generated section against the high-level plot plan. 3. **Creativity and Quality:** While harder to automate, aspects of creativity, prose quality, and character believability can be evaluated through human-in-the-loop feedback systems. Users interacting with the **novelscript-backend** could be prompted to rate the quality of generated scenes, providing valuable training data for future model tuning. 4. **RAG/GraphRAG Effectiveness:** Specific tests can be designed to probe the retrieval systems. For instance, a test case could introduce a key plot point in an early chapter and then later ask a question that requires knowing that detail. The system's ability to retrieve the correct chapter fragment (for RAG) or trace the correct relationship path (for GraphRAG) can be measured directly. The effectiveness of the hybrid retrieval can be tested by posing complex, multi-faceted questions that require synthesizing information from both sources.

By systematically addressing the challenges of data population, dynamic updates, and evaluation, the **novelscript-backend** can move from a promising architectural concept to a robust and reliable tool for collaborative storytelling. These operational pillars ensure that the sophisticated retrieval mechanisms are continuously fed with accurate information and that the overall system performance is constantly monitored and improved.

Reference

1. [https://cdn.qwenlm.ai/e7361f95-7d4f-473e-ad65-84221e19bff7/10488437-0d37-48aa-9225-9e17c2ad57a1_SRS_需求规格说明书.md?](https://cdn.qwenlm.ai/e7361f95-7d4f-473e-ad65-84221e19bff7/10488437-0d37-48aa-9225-9e17c2ad57a1_SRS_需求规格说明书.md?key=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJyZXNvdXJjZV91c2VyX2lkIjoizTczNjFmOTUtN2Q0Zi00NzNlWFknJUtODQyMjFlMTliZmY3IiwicmVzb3VyY2VfaWQiOiIxMDQ0ODQzNy0wZDM3LTQ4YWVlOTIyNS05ZTE3YzJhZDU3YTEiLCJyZXNvdXJjZV9jaGF0X2lkIjpudWxsQ.upl1rAQB3l0Ad1BCErooMEdG9VnGHiono506uSChMYs)
key=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJyZXNvdXJjZV91c2VyX2lkIjoizTczNjFmOTUtN2Q0Zi00NzNlWFknJUtODQyMjFlMTliZmY3IiwicmVzb3VyY2VfaWQiOiIxMDQ0ODQzNy0wZDM3LTQ4YWVlOTIyNS05ZTE3YzJhZDU3YTEiLCJyZXNvdXJjZV9jaGF0X2lkIjpudWxsQ.upl1rAQB3l0Ad1BCErooMEdG9VnGHiono506uSChMYs
2. https://cdn.qwenlm.ai/e7361f95-7d4f-473e-ad65-84221e19bff7/69e05c79-eb25-48d6-be9c-3c5883fbc913_SDS_软件设计说明书.md?

key=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJyZXNvdXJjZV91c2VyX2lkIjoizTczNjFmOTUtN2Q0Zi00NzNlWFkNjUtODQyMjFlMTliZmY3IiwicmVzb3VyY2VfaWQiOiI2OUWUwNWm3OS1lYjI1LTQ4ZDYtYmU5Yy0zYzU4ODNmYmM5MTMiLCJyZXNvdXJjZV9jaGF0X2lkIjpudWxsQ.jjZUKghYtalgTfWXltjO8mKSkiADc5JGk3ST_JFhIu4

3. Use pgvector for Vector Similarity Search <https://cloudberry.apache.org/docs/advanced-analytics/pgvector-search/>
4. Optimize generative AI applications with pgvector indexing - AWS <https://aws.amazon.com/blogs/database/optimize-generative-ai-applications-with-pgvector-indexing-a-deep-dive-into-ivfflat-and-hnsw-techniques/>
5. Combining pgvector and Apache AGE - knowledge graph ... <https://techcommunity.microsoft.com/blog/adforpostgresql/combining-pgvector-and-apache-age---knowledge-graph--semantic-intelligence-in-a-/4508781>
6. 在Azure Database for PostgreSQL 中使用pgvector 时优化性能 <https://docs.azure.cn/zh-cn/postgresql/extensions/how-to-optimize-performance-pgvector>
7. 零基础LangGraph 多智能体开发原创 - CSDN博客 https://blog.csdn.net/qq_41739364/article/details/146949124
8. Accelerate HNSW indexing and searching with pgvector on Amazon ... <https://aws.amazon.com/blogs/database/accelerate-hnsw-indexing-and-searching-with-pgvector-on-amazon-aurora-postgresql-compatible-edition-and-amazon-rds-for-postgresql/>
9. GraphRAG and PostgreSQL integration in docker with Cypher query ... <https://techcommunity.microsoft.com/blog/azure-ai-foundry-blog/graphrag-and-postgresql-integration-in-docker-with-cypher-query-and-ai-agents/4420623>
10. Optimize performance when using pgvector in Azure Database for ... <https://learn.microsoft.com/en-us/azure/postgresql/extensions/how-to-optimize-performance-pgvector>
11. Use HNSW index on Azure Cosmos DB for PostgreSQL for similarity ... <https://dev.to/sfoteini/use-hnsw-index-on-azure-cosmos-db-for-postgresql-for-similarity-search-5chk>
12. How to optimize performance when using pgvector - Azure Cosmos ... <https://learn.microsoft.com/en-us/azure/cosmos-db/postgresql/howto-optimize-performance-pgvector>
13. pgvector-study:基于Postgres 的向量相似性搜索扩展项目 - GitCode <https://gitcode.com/toadb/pgvector-study>